

Escalabilidad y rendimiento

Guía de referencia: qué hace falta para procesar más rápido, sin importar la escala

Proyecto: CamRecon Documento de referencia (no implementado)

ÍNDICE

1. Diagnóstico: por qué hoy es rápido y cuándo dejaría de serlo
2. Índice de vectores para la búsqueda de "quién es"
3. GPU y modelos de reconocimiento más modernos
4. Arquitectura de colas para muchas cámaras en simultáneo
5. Base de datos: de SQLite a PostgreSQL
6. Resumen y orden sugerido

Este documento es una **guía de referencia**, no un cambio ya implementado. Describe los cuellos de botella reales del sistema tal como está hoy, y qué haría falta en cada caso para que el reconocimiento sea rápido (o casi instantáneo) sin importar cuántas cámaras, empleados o rostros por imagen haya.

No hay un único cambio que resuelva todo de una vez — es una escalera: cada nivel cuesta más (en infraestructura, tiempo de desarrollo y, en algunos casos, dinero) pero también rinde más. Para el estado actual del proyecto (pocas cámaras, pocos empleados) **no hace falta nada de esto todavía**. Sirve como hoja de ruta para cuando el sistema crezca de verdad.

Diagnóstico: por qué hoy es rápido y cuándo dejaría de serlo

Con la arquitectura actual (`face_recognition` / `dlib` en CPU, comparación lineal contra los conocidos, SQLite, un hilo por cámara), el sistema es rápido porque:

- Hay pocas cámaras (paralelizadas con hilos, ver `tracking/service.py`).
- Hay pocos empleados cargados (la comparación es lineal: cada rostro detectado se compara contra **todos** los conocidos, uno por uno).
- El muestreo es cada pocos segundos, no frame por frame.

Esto deja de ser instantáneo cuando crece cualquiera de estas tres variables: cantidad de empleados conocidos, cantidad de cámaras simultáneas, o exigencia de latencia (tiempo real estricto en vez de

muestreo cada N segundos).

PUNTO 1

Índice de vectores para la búsqueda de "quién es"

EL PROBLEMA

Hoy, en `recognition.py`, `reconocer()` hace esto por cada rostro detectado:

```
distancias = face_recognition.face_distance(self.encodings_conocidos, encoding)
mejor_indice = int(np.argmin(distancias))
```

Es una **búsqueda lineal**: compara contra los N empleados conocidos, uno por uno. Con 10 empleados no se nota. Con 5.000, cada rostro detectado tarda ~500 veces más que hoy — el tiempo crece proporcional a la cantidad de gente cargada en el sistema.

LA SOLUCIÓN

Reemplazar la búsqueda lineal por un **índice de vectores** (vector search), que encuentra "el más parecido" en tiempo casi constante en vez de recorrer todo. Esto hace que el reconocimiento sea rápido **sin importar cuántos empleados tengas cargados**.

OPCIONES CONCRETAS

Opción	Cuándo conviene
FAISS (librería de Meta, corre embebida en el proceso Python)	Más simple de integrar, no requiere un servidor aparte. Ideal como primer paso.
pgvector (extensión de PostgreSQL)	Si de todas formas se migra a PostgreSQL (punto 4), resuelve este problema y el de la base de datos a la vez.
Qdrant / Milvus (bases de datos de vectores dedicadas)	Cuando se necesita buscar entre millones de rostros, con filtros adicionales, o servir la búsqueda desde varios servicios.

INSTRUCCIONES (CON FAISS, LA OPCIÓN MÁS SIMPLE)

- `pip install faiss-cpu` (o `faiss-gpu` si ya se tiene GPU, ver punto 2).
- En `recognition.py`, al cargar `conocidos/`, en vez de guardar `encodings_conocidos` como una lista simple, construir un índice:

```
import faiss
indice = faiss.IndexFlatL2(128) # 128 = dimensión del encoding de face_recognition
indice.add(np.array(encodings_conocidos, dtype="float32"))
```

3. En `reconocer()`, reemplazar el cálculo de `face_distance` por:

```
distancias, indices = indice.search(np.array([encoding], dtype="float32"), k=1)
```

4. Reconstruir el índice cada vez que se agrega/elimina un empleado (ya existe el punto de enganche: `recognizer.cargar_conocidos()` se llama en `empleados.py` cada vez que cambia algo — ahí mismo se reconstruye el índice).

IMPORTANTE

`IndexFlatL2` sigue siendo búsqueda exacta (no aproximada): ya es mucho más rápida que el bucle en Python porque está optimizada en C++/SIMD, pero para **millones** de rostros conviene un índice aproximado (`IndexIVFFlat`, `IndexHNSW`) — un poco menos preciso, mucho más rápido. Reconstruir el índice completo en cada alta/baja de empleado es aceptable hasta unos cuantos miles de personas; a mayor escala conviene un índice que soporte agregar/quitar vectores sin reconstruir todo. Este cambio **no requiere GPU ni cambiar el modelo de reconocimiento** — es el más barato de la lista y el que recomendaría primero si el proyecto crece.

PUNTO 2

GPU y modelos de reconocimiento más modernos

EL PROBLEMA

`face_recognition` / `dlib` corre en **CPU**. La detección de rostros y el cálculo de cada encoding son las operaciones más pesadas del pipeline, y en CPU tienen un techo de velocidad bajo — más notorio cuantas más caras haya en un mismo frame o cuantas más cámaras se procesen en paralelo.

LA SOLUCIÓN

Migrar a modelos de reconocimiento facial más modernos, corriendo en **GPU** vía ONNX Runtime o TensorRT. Los más usados en la industria hoy son variantes de **ArcFace/InsightFace** para generar los encodings, con **RetinaFace** o similar para la detección. Son más rápidos **y** más precisos que dlib.

INSTRUCCIONES

1. Conseguir un servidor/VPS con GPU NVIDIA (o una instancia cloud con GPU — AWS `g4dn` / `g5`, GCP con T4/L4, o un servidor propio con una GPU de gama media alcanza).
2. Instalar drivers NVIDIA + CUDA + cuDNN compatibles con la GPU.

3. Reemplazar `face_recognition` por `insightface` (`pip install insightface onnxruntime-gpu`) o un pipeline equivalente en ONNX Runtime/TensorRT.
4. Reescribir `recognition.py` para usar la API del nuevo modelo (detección + extracción de embedding) en vez de `face_recognition.face_locations` / `face_encodings`.
5. **Volver a generar los encodings de todos los empleados ya cargados** (ver consideración más abajo).
6. Si se procesan varias cámaras a la vez, agrupar los frames en **lotes (batching)** antes de mandarlos al modelo — las GPU rinden mucho mejor procesando varias imágenes juntas que una por una.

IMPORTANTE

Los encodings de un modelo **no son compatibles** con los de otro. Los vectores que genera dlib no se pueden comparar contra los que genera ArcFace — son espacios matemáticos distintos. Migrar de modelo implica **volver a procesar todas las fotos de referencia** de **conocidos/** (no se pierden las fotos, pero sí hay que recalcular los encodings guardados). Además tiene un costo de infraestructura real: una GPU dedicada es más cara que un VPS común — vale la pena a partir de cierta escala, no antes. Es el cambio que **más impacto tiene en velocidad pura** (mejoras de 10x a 50x son típicas), pero también el que más esfuerzo de desarrollo/infraestructura lleva de los cuatro.

PUNTO 3

Arquitectura de colas para muchas cámaras en simultáneo

EL PROBLEMA

Hoy, `tracking/service.py` corre un hilo por cámara **dentro del mismo proceso** del backend. Funciona bien hasta cierto punto (unas cuantas decenas de cámaras, según CPU disponible), pero todo compite por los mismos recursos del único servidor, y si el proceso se cae, se cae todo el tracking junto con la API.

LA SOLUCIÓN

Desacoplar la **captura** de frames del **procesamiento** (detección + reconocimiento) usando una cola de mensajes, y tener varios **workers** (procesos o contenedores independientes) consumiendo de esa cola en paralelo. Así se puede escalar horizontalmente — agregar más workers cuando hacen falta — sin tocar el resto del sistema.

[illegible]

INSTRUCCIONES

1. Levantar un broker de mensajes: **Redis** (más simple, `redis-server` o contenedor Docker) o **RabbitMQ** (más robusto para colas complejas).
2. Elegir un framework de tareas en background: **Celery** (el más usado con Python) o **RQ** (más simple, basado en Redis) o **Arq** (async, también sobre Redis).
3. Separar el proceso actual en dos roles:
 - **Productor**: sigue conectándose a cada cámara por RTSP y captura frames, pero en vez de procesarlos ahí mismo, los publica en la cola (como bytes JPEG + metadata de la cámara).
 - **Worker(s)**: procesos separados (se pueden correr varias instancias, incluso en máquinas distintas) que toman frames de la cola, corren `recognizer.reconocer()`, y escriben el resultado en la base de datos.
4. Contenerizar con Docker Compose, con un servicio por worker y `replicas` para escalar la cantidad según necesidad.

IMPORTANTE

El `session_tracker.py` actual asume que se procesa de a una detección por vez para decidir si abre/cierra una sesión. Con varios workers escribiendo en paralelo hay que cuidar las condiciones de carrera (dos workers detectando a la misma persona casi al mismo tiempo en cámaras distintas) — conviene un lock por persona (ej. un lock distribuido en Redis) al momento de abrir/cerrar sesiones. Este punto **depende del punto 4** (base de datos): SQLite no soporta bien escrituras concurrentes desde varios procesos a la vez, así que antes de escalar a múltiples workers conviene migrar a PostgreSQL. También suma una pieza de infraestructura nueva (el broker) que hay que operar y monitorear.

PUNTO 4

Base de datos: de SQLite a PostgreSQL

EL PROBLEMA

SQLite (lo que usa el sistema hoy, `backend/tracking.db`) permite **un solo escritor a la vez**. Es ideal para esta prueba de concepto por su simplicidad (un solo archivo, cero configuración), pero se vuelve un cuello de botella real en cuanto hay varias cámaras o workers escribiendo detecciones al mismo tiempo.

LA SOLUCIÓN

Migrar a **PostgreSQL**, que sí soporta múltiples conexiones escribiendo en paralelo de forma segura. De paso, instalando la extensión `pgvector`, PostgreSQL puede resolver también el punto 1 (búsqueda de vectores) en la misma base, sin sumar otro sistema aparte.

INSTRUCCIONES

1. Levantar un servidor PostgreSQL (VPS propio, contenedor Docker, o un servicio administrado como RDS/Cloud SQL/Supabase).
2. Instalar la extensión `pgvector` si también se va a resolver el punto 1 ahí: `CREATE EXTENSION vector;`
3. Instalar el driver Python: `pip install psycopg2-binary` (o `asyncpg` si se migra a un stack async).
4. Reescribir `tracking/db.py`: cambiar la conexión de `sqlite3.connect(...)` al driver de Postgres, y ajustar las pocas diferencias de sintaxis SQL entre SQLite y Postgres (tipos de datos, `AUTOINCREMENT` → `SERIAL` / `IDENTITY`, `ON CONFLICT` tiene una sintaxis un poco distinta, etc.). El resto de la lógica no debería necesitar cambios porque ya está separada del detalle de SQL.
5. Migrar los datos existentes (`tracking.db` → Postgres) con un script simple que lea cada tabla de SQLite y la inserte en Postgres.
6. Sumar un **pool de conexiones** (ej. `psycopg2.pool` o PgBouncer delante de Postgres) para que muchos workers concurrentes no agoten las conexiones disponibles.

IMPORTANTE

Es el cambio con **menor riesgo técnico** de los cuatro (es una migración de base de datos bien conocida, no un cambio de algoritmo ni de infraestructura exótica), pero agrega una pieza más para operar: hay que hacer backups, monitorear el servidor, gestionar accesos. Si se planea implementar el punto 3, este punto es **prerrequisito** — sin esto, escalar workers solo va a mover el cuello de botella a la base de datos. No hace falta migrar todo de una — se puede hacer en un momento de mantenimiento planificado, con la base actual de SQLite como respaldo hasta confirmar que todo funciona igual en Postgres.

Resumen y orden sugerido

#	Cambio	Impacto en velocidad	Costo/esfuerzo	Cuándo se justifica
1	Índice de vectores (FAISS)	Alto cuando hay muchos empleados	Bajo	Cientos/miles de empleados cargados
4	PostgreSQL	Medio (destraba el punto 3)	Medio	Varias cámaras/workers escribiendo a la vez
3	Colas + workers	Alto en throughput con muchas cámaras	Medio-alto	Decenas de cámaras en simultáneo
2	GPU + modelo moderno	Muy alto por imagen procesada	Alto (infraestructura + re-enrolar a todos)	Necesidad real de latencia casi nula, o volumen muy alto de cámaras/imágenes

En una frase

No hace falta implementar los cuatro puntos para tener un sistema notablemente más rápido — el punto 1 por sí solo, bien implementado, ya resuelve el escenario más común (crece la cantidad de empleados, no la cantidad de cámaras). Los puntos 3 y 4 se justifican cuando crece la cantidad de **cámaras simultáneas**. El punto 2 es el salto más grande y el único que conviene evaluar solo cuando los otros tres ya no alcancen.